Instituto Federal de Santa Catarina –
Campus Florianópolis
Departamento Acadêmico de Eletrônica

Programação C++
Prof. Matheus Leitzke Pinto matheus.pinto@ifsc.edu.br

Lista de Exercícios –

Introdução a classes e objetos

Nota: Para todos os exercícios nesta lista, utilize os conceitos de encapsulamento. Ou seja, atributos devem ser privados, e qualquer modificação ou recuperação dos valores devem ser feitos por meio de métodos do tipo *getter* e *setter*.

1) Crie uma classe chamada "Point" que represente as coordenadas (x, y) de um ponto no plano cartesiano. Implemente métodos para definir e obter as coordenadas.

2) Crie uma classe "Rectangle" que tenha dois objetos da classe " Point" como vértices opostos do retângulo. Implemente métodos para calcular a área e o perímetro do retângulo. Também, crie um método que receba um objeto Point e verifique se ele se encontra dentro do retângulo.

3) Crie uma classe "Circle" que tenha um objeto da classe " Point" como centro do círculo, e que receba no seu construtor o raio do círculo. Implemente métodos para calcular a área e o perímetro do círculo. Também, crie um método que receba um objeto Point e verifique se ele se encontra dentro do círculo.

4) Crie uma classe "Triangle" que representa um triangulo. Os vértices do triangulo são definidos por três objetos "Point", passados no construtor. A classe deve implementar um método para verificar se é um triângulo válido e outro para calcular o tipo do triângulo (equilátero, isósceles ou escaleno) e calcular a área do triangulo. Também deve possuir um método que recebe outro objeto Triangle e retorna true se os objetos são iguais, ou false caso contrário.

5) Crie uma classe "GeometryVerifier" que recebe dois objetos geométricos (Circle, Rectangle ou Triangle) e retorna "verdadeiro" se objeto cabe completamente dentro do outro, e "falso" caso contrário.

6) Crie uma classe "BankAccount" com os atributos: número da conta, saldo e nome do titular. Implemente métodos para depositar, sacar e obter o saldo atual. Não chame cin e cout dentro dos métodos.

- `void DepositCash(float amount)` – que recebe a quantidade de dinheiro em reais (`amount`), e realiza o deposito do valor.
- `int WithdrawCash(uint32_t pwd, float amount)` – se a senha (`pwd`) estiver correta, permite o saque de dinheiro em reais requisitado (`amount`). Caso o valor requisitado seja maior que o saldo disponível, deve retornar o valor enumerado "`INSUFFICIENT_BALANCE = 2`". Caso a senha esteja incorreta, deve retornar o valor enumerado "`INCORRECT_PWD = 1`". Caso o saque tenha sido permitido, deve retornar o valor enumerado "`TRANSACTION_OK = 0`".
- `int CheckAccount(uint32_t pwd, float &amount)` – se a senha (`pwd`) estiver correta, retorna por referência a quantidade de dinheiro em reais depositada na conta (`amount`). Caso a senha esteja incorreta, deve retornar o valor enumerado "`INCORRECT_PWD = 1`". Caso a senha esteja correta, deve retornar o valor enumerado "`TRANSACTION_OK = 0`".

7) Crie uma classe "TerminalBank" que receberá uma conta e irá interagir com o usuário através das operações cin e cout. A classe deve possuir os seguintes métodos:

- `void ConnectAccount(BankAccount &account)` – que define a conta que será utilizada no terminal.
- `int DepositCash()` – que irá pedir ao usuário, a quantidade de dinheiro em reais, a senha da conta, realizando o deposito do valor. Caso nenhuma conta esteja conectada, deve retornar o valor enumerado "`NO_ACCOUNT_CONNECTION = 3`". Caso o deposito tenha sido feito, deve retornar o valor enumerado "`TRANSACTION_OK = 0`", e a função deve imprimir na saída padrão "Valor depositado: <quantidade_depositada>".
- `int WithdrawCash()` – que deve retirar da conta, a quantidade de dinheiro em reais requisitada. Caso o valor requisitado seja maior que o saldo disponível, deve retornar o valor enumerado "`INSUFFICIENT_BALANCE = 2`". Caso a senha esteja incorreta, deve retornar o valor enumerado "`INCORRECT_PWD = 1`". Caso nenhuma conta esteja conectada, deve retornar o valor enumerado "`NO_ACCOUNT_CONNECTION = 3`". Caso o deposito tenha sido feito, deve retornar o valor enumerado "`TRANSACTION_OK = 0`".
- `int CheckAccount()` – que imprime a quantidade de dinheiro em reais depositada na conta. Caso a senha esteja incorreta, deve retornar o valor enumerado "`INCORRECT_PWD = 1`". Caso a senha esteja correta, deve retornar o valor enumerado "`TRANSACTION_OK = 0`".
- `void DisconnectAccount()` – que desconecta a conta do terminal, fazendo com que nenhuma transição seja possível.

O programa irá executar indefinidamente e irá encerrar apenas quando forçado na IDE

8) Crie uma classe “DiffRobot” que controle a velocidade de um robô diferencial. A classe deve possuir os seguintes métodos:

- `int SetTwist(float v, float w)` – que recebe as velocidades linear (v), em m/s, e angular (ω), em rad/s e define as velocidades de atuação nas duas rodas. A função deve retornar um de dois possíveis valores enumerados: 'STATUS_OK = 0', se calculo ocorreu como esperado, ou 'STATUS_SPEED_OVF = 1', caso o valor calculado para uma das rodas tenha passado do limite estabelecido.
- `void GetTwist(float &v, float &w)` – deve retornar por meio de passagem por referência as velocidades linear (v), em m/s, e angular (ω), em rad/s.
- `void Stop()` – deve fazer com que o robô pare, fazendo com que as rodas não rotacionem.
- `void GetMotorsSpeed(float &v1, float &v2)` – retorna por referência, as velocidades dos motores em m/s;

As equações para o cálculo da cinemática inversa são dadas por:

$$v_1 = v + l\omega$$

$$v_2 = v - l\omega$$

Onde: v_1 é a velocidade da roda esquerda; v_2 é a velocidade da roda direita e; l é a largura do eixo do robô.

A largura l deve ser definida no início do programa, através da entrada padrão acompanhada da mensagem “Insira a largura do eixo: ”. Após isso, o usuário deve passar a máxima velocidade linear das rodas do robô, com as seguintes mensagens: “Insira a velocidade máxima das rodas: ”.
Após isso, deve ser apresentado um menu, para que o usuário possa escolher o que fazer:

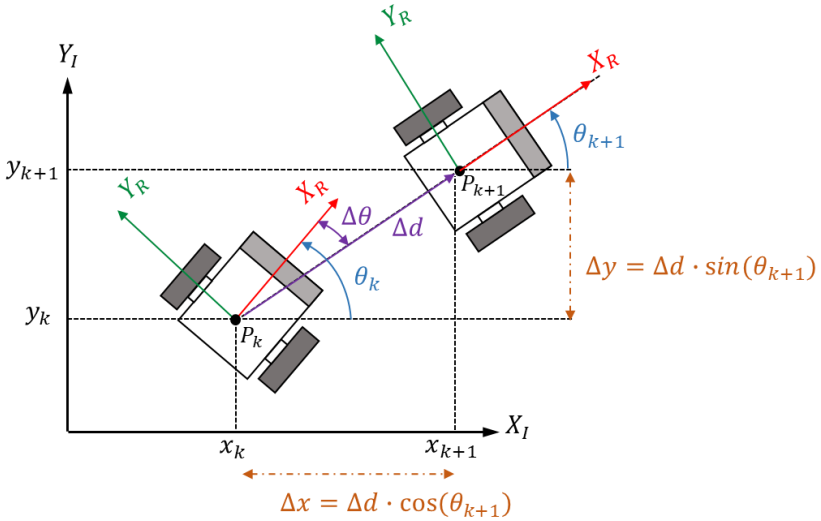
```
Escolha a opção:

1 – Definir o twist

2 – Verificar o twist

3 – Verificar a velocidade das rodas

4 – Parar o robô
```

Dever ser passado um dos números inteiros apresentado no menu (1, 2, ou 3). Caso seja passado um valor errado de twist ou uma opção errada de menu, deve ser enviada a mensagem "Valor não definido". O menu, sempre deve ser reapresentado após uma ação for concluída. O programa irá encerrar apenas quando forçado na IDE.

9) Acrescente à classe do exercício anterior, a função para obter a posição cartesiana e ângulo do robô (pose do robô) no ambiente em que está se locomovendo:

- `void GetPose(float &x, float &y, float &theta)` – retorna a posição cartesiana (x,y) do robô e seu ângulo θ em relação ao eixo X.

Os parâmetros do robô podem ser observados conforme figura abaixo:



Você deve passar a pose inicial do robô durante a inicialização do programa, juntamente com os parâmetros de máxima velocidade das rodas.

Considere que o robô irá se locomover por 2 segundos a cada chamada de SetTwist. Dessa forma, o deslocamento Δd do robô em relação ao seu ponto P no instante k (P_k) e o deslocamento de ângulo $\Delta\theta$ no plano cartesiano no intervalo $\Delta t = 2s$ é dado por:

$$\Delta d = v \cdot \Delta t$$

$$\Delta\theta = \omega \cdot \Delta t$$

Portanto, a pose do robô no instante k+1 pode ser obtida a partir das seguintes equações:

$$x_{k+1} = x_k + \Delta x$$

$$y_{k+1} = y_k + \Delta y$$

$$\theta_{k+1} = \theta_k + \Delta\theta$$

O menu deve ser atualizado da seguinte forma:

```
Escolha a opção:

1 - Definir o twist

2 - Verificar o twist

3 - Verificar a velocidade das rodas

4 - Verificar a pose

5 - Parar o robô
```